Royaume du Maroc

Université Sultan Moulay Slimane

Ecole Supérieure de Technologie – Béni Mellal

Département Informatique et Technologies de Gestion

Rapport Académique de Projet de Fin de Module

ClientLedger

Architecture Cloud-Native & Sécurité IAM sur AWS

Réalisé par	Mohamed Khalis & Intissar Farhoun
Encadré par	S .Mahrach
Établissement	ESTBM — École Supérieure de Technologie de Béni Mellal
Filière	Bachelor Cybersécurité
Année universitaire	2025 / 2026
Dépôt GitHub	github.com/Admiral-Simo/ClientLedger

Remerciements

Nous tenons à exprimer notre profonde gratitude à M .Safae Mahrach, notre encadrant pédagogique, pour ses orientations précieuses et son suivi rigoureux tout au long de ce projet de fin d'études.

Nos remerciements vont également à l'ensemble du corps enseignant de l'École Supérieure de Technologie de Béni Mellal, qui nous a fourni les bases théoriques indispensables à la réalisation de ce travail, ainsi qu'à notre famille et nos proches pour leur soutien moral constant.

Ce projet a mobilisé des compétences en génie logiciel, en sécurité informatique et en infrastructure cloud. Nous remercions la communauté open-source et la documentation officielle AWS et Spring Boot, qui ont constitué des ressources essentielles.

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
EC2	Elastic Compute Cloud
HTTP(S)	HyperText Transfer Protocol (Secure)
IAM	Identity and Access Management
IaC	Infrastructure as Code
IDOR	Insecure Direct Object Reference
IoC	Inversion of Control
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
JWKS	JSON Web Key Set
KID	Key Identifier
OWASP	Open Web Application Security Project
PCI-DSS	Payment Card Industry – Data Security Standard
RBAC	Role-Based Access Control
RDS	Relational Database Service
REST	Representational State Transfer
RSA	Rivest–Shamir–Adleman (algorithme cryptographique)
SaaS	Software as a Service
SES	Simple Email Service
SLA	Service Level Agreement
SPOF	Single Point of Failure
SRP	Secure Remote Password
SSR	Server-Side Rendering
TLS	Transport Layer Security
TOC	Table of Contents
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
VPC	Virtual Private Cloud

Table des Matières

Remerciements 2
Liste des Abréviations 3
Table des Matières 4
Liste des Figures et Tableaux 6
Introduction Générale 7
Chapitre 1 : Introduction Générale & Contexte du Projet 8
1.1 Contexte et Motivation 8
1.2 Présentation de ClientLedger 8
1.3 Objectifs du Projet 8
1.4 Périmètre Fonctionnel 9
1.5 Étude Non Fonctionnelle 9
Chapitre 2 : Stack Technologique 11
2.1 Vue d'Ensemble des Technologies 11
2.2 Technologies Frontend 11
2.3 Technologies Backend 13
2.4 Infrastructure & Cloud 14
Conclusion 15
Chapitre 3 : Architecture Système & Modélisation des Données 16
3.1 Paradigme Client-Serveur Stateless 16
3.2 Modèle Entité-Relation 16
3.2.1 Entité USERS — Identité Cognito 16
3.2.2 Entité CLIENTS — Gestion des Tiers 17
3.2.3 Entité CONTRACTS — Cycle de Vie Contractuel 17
3.3 Diagramme de Classes UML Complet 18
Chapitre 4 : Infrastructure Cloud AWS 20
4.1 Topologie de Déploiement 20
4.2 Composants AWS Déployés 21
4.2.1 AWS API Gateway (HTTP API) 21
4.2.2 AWS Elastic Beanstalk (Backend) 21
4.2.3 Amazon S3 + API Gateway (Frontend) 21
4.2.4 Amazon RDS (MySQL 8.0) 21
4.2.5 Amazon SES (Simple Email Service) 22
4.3 Pipeline CI/CD avec GitHub Actions 22
5.1 Architecture Zero-Trust 23
5.2 Authentification Déléguée via AWS Cognito 23
5.3 Vérification JWT & JWKS 23
5.4 Isolement Multi-Tenant 24
5.5 Tableau de Synthèse des Contrôles de Sécurité 24

Chapitre 6 : Architecture Applicative & API REST 26
6.1 Architecture en Couches du Backend 26
6.2 Endpoints de l'API REST 26
6.3 Gestion des Exceptions Globale 27
6.4 Génération de PDF Contractuels 27
Chapitre 7 : Réalisation 28
7.1 Page d'Accueil — Landing Page 28
7.2 Page de Tarification 28
7.3 Portail de Connexion Sécurisée 30
7.4 Dashboard Financier Principal 30
7.5 Gestion des Contrats 31
7.6 Création d'un Nouveau Contrat 32
7.7 Paramètres et Profil de l'Entreprise 33
7.8 Génération et Aperçu de Factures PDF 34
Chapitre 8 : Analyse, Évaluation & Perspectives 36
8.1 Analyse des Forces du Projet 36
8.2 Feuille de Route Future 36
7.2.1 Infrastructure de Paiement Automatisée (Stripe) 36
8.2.2 Atténuation des Menaces Avancées (AWS WAF & GuardDuty) 36
8.2.3 Observabilité & SRE 37
Conclusion Générale 38
Bibliographie & Références 39
Documentation Officielle 39
Articles & Standards 39
Dépôt du Projet 39

Liste des Figures et Tableaux

Page d'accueil de ClientLedger : accroche et navigation principale 28
Page de tarification : plans Starter Beta ($0) et Pro Ledger ($15/mois) 29
Portail de connexion sécurisé avec authentification AWS Cognito 30
Dashboard financier : KPIs, Revenue Trends et Financial Health 31
nterface de gestion des contrats avec filtres, badges de statut et pagination 32
Modale de création d'un nouveau contrat avec validation de formulaire 33
Page de paramètres : configuration du profil entreprise et des informations de facturation 34
Aperçu d'une facture PDF générée par ClientLedger 34

Introduction Générale

Le secteur du travail indépendant (freelancing) connaît une transformation profonde à l'échelle mondiale. Les plateformes numériques ont démocratisé l'accès aux clients internationaux, permettant à des développeurs, designers et consultants de collaborer à distance avec des entreprises du monde entier. Cependant, cette croissance s'accompagne d'une réalité administrative souvent négligée : la gestion rigoureuse des contrats, des paiements et des relations clients.

Face à cette problématique, ClientLedger se positionne comme une solution SaaS (Software as a Service) cloud-native, conçue et déployée intégralement sur Amazon Web Services. Ce projet de fin d'études illustre la convergence entre deux disciplines complémentaires enseignées au sein du Bachelor Cybersécurité de l'ESTBM : le génie logiciel cloud et la sécurité des systèmes d'information.

Ce rapport académique documente l'ensemble du cycle de vie du projet, depuis l'analyse des besoins jusqu'aux perspectives d'évolution, en passant par les choix architecturaux, la modélisation des données, l'infrastructure AWS, et les mécanismes de sécurité IAM déployés. Chaque décision technique est justifiée par des principes d'ingénierie éprouvés et des standards de l'industrie (NIST, OWASP, RFC).

Le rapport est structuré en sept chapitres thématiques : le premier pose le contexte et les objectifs du projet ; le deuxième présente la stack technologique retenue ; le troisième détaille la modélisation des données ; le quatrième décrit l'infrastructure AWS provisionnée par Terraform ; le cinquième analyse les mécanismes de cybersécurité et d'IAM ; le sixième présente l'architecture applicative et l'API REST ; et le septième propose une analyse critique et une feuille de route prospective.

Chapitre 1 : Introduction Générale & Contexte du Projet

Introduction

Ce chapitre établit le contexte du projet ClientLedger en analysant la problématique opérationnelle des freelancers, en définissant les objectifs fonctionnels et non fonctionnels, et en identifiant les parties prenantes du système. Il pose les fondements qui justifient l'ensemble des choix architecturaux et technologiques documentés dans les chapitres suivants.

1.1 Contexte et Motivation

Le marché mondial du freelancing connaît une croissance exponentielle. Selon diverses études sectorielles, plus de 50 % des travailleurs indépendants avouent avoir perdu des revenus à cause de factures impayées mal suivies ou de litiges contractuels dus à des accords informels. La réalité opérationnelle du freelancer moderne est caractérisée par trois problèmes structurels majeurs :

- Fuites de revenus : des paiements oubliés ou des relances tardives entraînent des pertes financières directes.
- Dérive de périmètre (Scope Creep) : des accords conclus sur des messageries instantanées génèrent des disputes coûteuses.
- Workflows non professionnels : la gestion de contrats complexes via des feuilles de calcul compromet la crédibilité professionnelle.

1.2 Présentation de ClientLedger

ClientLedger est une solution SaaS (Software as a Service) cloud-native développée pour centraliser et professionnaliser la gestion administrative des freelancers. Il se positionne comme une Source Unique de Vérité pour chaque interaction client, combinant les fonctions d'un gestionnaire de cycle de vie des contrats et d'un moniteur de santé financière.

L'application est hébergée intégralement sur Amazon Web Services (AWS) et son infrastructure est codifiée via Terraform, garantissant une reproductibilité complète de l'environnement de déploiement. L'authentification et la gestion des identités sont déléguées à AWS Cognito, conférant à la plateforme une sécurité de niveau entreprise sans les risques liés au stockage local de credentials.

1.3 Objectifs du Projet

- Concevoir et déployer une architecture cloud-native multi-couches sur AWS.
- Implémenter un système IAM Zero-Trust basé sur JWT RS256 et AWS Cognito.
- Garantir l'isolement des données multi-tenant via des filtres cryptographiques owner_id.

- Automatiser le provisioning de l'infrastructure via Terraform (IaC).
- Mettre en place un pipeline CI/CD avec GitHub Actions pour un déploiement continu.
- Fournir un reporting financier en temps réel aux freelancers via un dashboard analytique.

1.4 Périmètre Fonctionnel

Le périmètre fonctionnel de ClientLedger couvre l'ensemble du cycle de vie administratif d'un freelancer : depuis l'enregistrement d'un nouveau client jusqu'à la clôture d'un contrat et la visualisation des indicateurs financiers.

- Gestion des clients : création, consultation, archivage avec support multi-devises.
- Gestion des contrats : cycle de vie complet (DRAFT → ACTIVE → COMPLETED / CANCELLED).
- Suivi des paiements : jalons de paiement avec statuts PENDING, PAID, OVERDUE.
- Dashboard financier : revenus mensuels, taux de recouvrement, contrats actifs.
- Notifications email : alertes automatiques via Amazon SES.

1.5 Étude Non Fonctionnelle

Les exigences non fonctionnelles définissent les contraintes de qualité que le système doit respecter indépendamment de ses fonctionnalités métier. Ces critères sont essentiels pour garantir la viabilité d'une application SaaS en production.

Exigence Non Fonctionnelle	Description & Critère de Succès
Performance	Temps de réponse API < 200ms pour 95% des requêtes en conditions normales de charge (p95 < 200ms).
Disponibilité	Objectif SLA de 99.9% (uptime mensuel). AWS Elastic Beanstalk avec health checks automatiques.
Scalabilité	Architecture stateless permettant le scaling horizontal. Elastic Beanstalk supporte l'auto-scaling.
Sécurité	Zero-Trust : chaque requête est authentifiée par JWT RS256. Données chiffrées en transit (TLS 1.2+).
Confidentialité	Isolement multi-tenant cryptographique par owner_id. Aucune donnée inter-utilisateurs accessible.
Maintenabilité	Architecture en couches (Controller/Service/Repository). Tests JPA. Coverage minimal requis : 70%.
Portabilité	Infrastructure codifiée via Terraform. Déployable sur tout compte AWS avec terraform apply.
Traçabilité	Audit trail sur toutes les entités (created_at, updated_at immuables via @CreationTimestamp).
Compatibilité	API REST standard JSON/HTTP. Compatible avec tout client HTTP (web, mobile, tiers).
Conformité	Architecture réduit le périmètre PCI-DSS. Prête pour intégration Stripe Checkout sans stockage CB.

Tableau 1.1 — Matrice des Exigences Non Fonctionnelles de ClientLedger

Conclusion du chapitre

Ce chapitre a posé les fondements du projet ClientLedger en analysant la problématique des freelancers, en définissant les objectifs fonctionnels (gestion des clients, contrats, paiements, dashboard) et en établissant une matrice complète des exigences non fonctionnelles (performance, sécurité, disponibilité, maintenabilité). Ces exigences guident directement les choix architecturaux des chapitres suivants.

Chapitre 2 : Stack Technologique

Introduction

Ce chapitre présente et justifie l'ensemble des technologies retenues pour le développement et le déploiement de ClientLedger. Pour chaque technologie, nous analysons son rôle spécifique dans l'architecture, ses avantages par rapport aux alternatives, et son intégration avec les autres composants du système.

2.1 Vue d'Ensemble des Technologies

ClientLedger adopte une architecture découplée où chaque couche applicative s'appuie sur des technologies éprouvées, choisies pour leur robustesse en environnement de production et leur adéquation avec les contraintes de sécurité d'un système multi-tenant.

Couche	Technologie	Rôle
Frontend	Next.js / React / TypeScript	Interface utilisateur SPA
Backend	Java 17 Spring Boot 3	API REST métier
Base de données	Amazon RDS (MySQL 8.0)	Persistance relationnelle
Authentification	AWS Cognito User Pools	IAM & JWT OIDC
Infrastructure	Terraform (IaC)	Provisioning AWS déclaratif
CI/CD	GitHub Actions	Déploiement automatisé
Hébergement Frontend	AWS S3 + API Gateway	Hosting statique sécurisé
Hébergement Backend	AWS Elastic Beanstalk	Orchestration EC2 t3.micro
Messagerie	Amazon SES	Notifications email transactionnelles

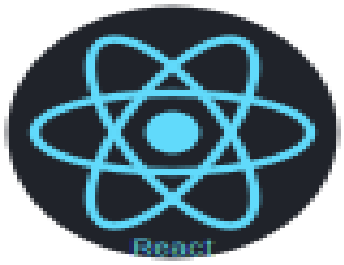
Tableau 2.1 — Stack Technologique Complète de ClientLedger

2.2 Technologies Frontend



Next.js 14 — Framework React Full-Stack

Next.js 14 constitue le socle du frontend de ClientLedger. Ce framework React permet un rendu hybride (Server-Side Rendering et Static Site Generation), offrant des performances optimales. Pour ClientLedger, le build statique est généré et hébergé sur AWS S3, éliminant tout coût d'infrastructure serveur frontend.



React 18 — Bibliothèque UI Composants

React 18 fournit le modèle de composants UI de l'application. Les hooks React (useState, useEffect, useContext) permettent une gestion d'état réactive et performante. L'écosystème React offre une large bibliothèque de composants réutilisables qui accélèrent le développement.



TypeScript — Typage Statique JavaScript

TypeScript apporte le typage statique à l'écosystème JavaScript. Cette couche de type permet la détection précoce des erreurs de type à la compilation plutôt qu'à l'exécution, améliore l'autocomplétion dans les IDEs, et facilite la maintenabilité du code dans la durée.



TailwindCSS — Framework CSS Utilitaire

TailwindCSS est utilisé comme framework CSS utilitaire, permettant un développement rapide d'interfaces cohérentes sans surcharge de code CSS personnalisé. Son système de purge automatique des classes inutilisées garantit un bundle CSS minimal en production.

2.3 Technologies Backend



Java 17 — Langage de Programmation

Java 17 est la version LTS (Long-Term Support) choisie pour le backend. Sa robustesse éprouvée en environnement enterprise, son écosystème de bibliothèques mature, et sa compatibilité native avec Spring Boot 3 en font le choix optimal pour une API REST sécurisée.



Spring Boot 3 — Framework Java Backend

Spring Boot 3 est le framework central du backend. Il fournit un conteneur IoC (Inversion of Control) avec injection de dépendances, une intégration native avec Spring Security pour la protection des endpoints via OAuth2 Resource Server, et Spring Data JPA pour l'abstraction de la couche de persistance relationnelle.

La configuration Spring Security est minimale grâce à l'autoconfiguration : l'annotation @EnableWebSecurity et la méthode oauth2ResourceServer(oauth2 -> oauth2.jwt()) suffisent à activer la validation JWT complète. Le filtre de vérification JWKS est géré automatiquement par la bibliothèque Nimbus JOSE+JWT.



Apache Maven

Maven est l'outil de build du projet backend. Il gère le téléchargement des dépendances depuis le dépôt central Maven, l'exécution des tests unitaires lors de la phase verify, et la génération du JAR exécutable déployé sur Elastic Beanstalk.

2.4 Infrastructure & Cloud



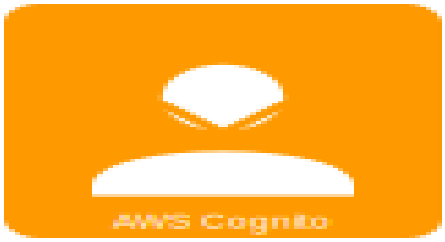
Terraform — Infrastructure as Code

Terraform est l'outil IaC (Infrastructure as Code) qui codifie l'intégralité de l'infrastructure AWS de ClientLedger. Son approche déclarative (on décrit l'état désiré, Terraform calcule les actions) garantit la reproductibilité totale et élimine la dérive de configuration entre les environnements.



MySQL 8.0 — Système de Gestion de Base de Données

MySQL 8.0 est le moteur de base de données relationnelle hébergé sur Amazon RDS. Les propriétés ACID garantissent l'intégrité des transactions financières. Le type DECIMAL(19,4) est utilisé pour les montants contractuels afin d'éviter les erreurs d'arrondi inhérentes aux types flottants.



AWS Cognito — Identity & Access Management

AWS Cognito User Pools gère l'authentification des utilisateurs de ClientLedger. Il émet des tokens JWT signés par cryptographie asymétrique RS256, fournit une protection anti-brute-force native, et expose un endpoint JWKS permettant au backend de vérifier les signatures sans stocker de secrets partagés.



Amazon RDS — Relational Database Service

Amazon RDS gère le cycle de vie opérationnel de la base de données MySQL : patches de sécurité automatiques, sauvegardes quotidiennes avec point-in-time recovery, et monitoring CloudWatch intégré. L'instance est déployée dans un sous-réseau privé du VPC, inaccessible depuis l'internet public.



GitHub Actions — CI/CD Pipeline

GitHub Actions automatise le pipeline de déploiement continu. Un workflow YAML déclenché sur chaque push sur la branche principale exécute séquentiellement : compilation Maven, tests unitaires, build Next.js, et déploiement sur les services AWS cibles via les credentials IAM stockés dans les GitHub Secrets chiffrés.

Conclusion

Ce chapitre a présenté et justifié l'ensemble de la stack technologique de ClientLedger. Le choix de Next.js/React/TypeScript pour le frontend assure des interfaces réactives et maintenables. Java Spring Boot 3 offre un backend robuste avec sécurité OAuth2 intégrée. Terraform garantit la reproductibilité de l'infrastructure AWS, tandis qu'AWS Cognito délègue la gestion des identités à un service managé certifié. Cette combinaison constitue une stack moderne, sécurisée et adaptée aux exigences d'un SaaS en production.

Chapitre 3 : Architecture Système & Modélisation des Données

Introduction

Ce chapitre décrit l'architecture applicative de ClientLedger et la modélisation du domaine métier. Il présente le paradigme client-serveur stateless, détaille les quatre entités JPA du modèle relationnel (Users, Clients, Contracts, Payments) avec leurs contraintes d'intégrité, et explique comment le mécanisme owner_id implémente l'isolement multi-tenant au niveau de la donnée.

3.1 Paradigme Client-Serveur Stateless

L'architecture de ClientLedger repose sur une séparation stricte des responsabilités entre la couche de présentation et la logique métier. Cette conception stateless présente plusieurs avantages fondamentaux pour un déploiement cloud :

- Scalabilité horizontale : chaque instance backend est indépendante de l'état de session.
- Déploiement blue/green simplifié : aucune session à migrer lors des mises à jour.
- Sécurité renforcée : l'état d'authentification est porté par le JWT, validé à chaque requête.

3.2 Modèle Entité-Relation

Le domaine métier de ClientLedger est structuré autour de quatre entités principales, normalisées pour éviter la redondance et optimiser les requêtes JPA via des opérations JOIN FETCH efficaces.

3.2.1 Entité USERS — Identité Cognito

L'entité USERS ne stocke pas de mot de passe. Elle persiste uniquement les métadonnées de profil et le mapping avec le Subject ID (sub) cryptographique émis par AWS Cognito.

USERS			
Type SQL	**Colonne**	**Clé**	**Description**
UUID	id	PK	Clé primaire v4 — mapped sur Cognito sub
VARCHAR	email	UK	Email unique, indexé
VARCHAR	password_hash		Bcrypt/Argon2 (local auth fallback)

VARCHAR	full_name		Nom complet du freelancer
TIMESTAMP	created_at		Audit trail — création
TIMESTAMP	updated_at		Audit trail — modification

Figure 3.1 — Diagramme UML : Entité USERS avec mapping JWT

3.2.2 Entité CLIENTS — Gestion des Tiers

L'entité CLIENTS représente les tiers commerciaux du freelancer. La colonne owner_id crée un lien cryptographique entre le client et son propriétaire Cognito, implémentant ainsi l'isolement multi-tenant au niveau de la donnée.

CLIENTS			
Type SQL	**Colonne**	**Clé**	**Description**
BIGINT	id	PK	Auto Increment
UUID	user_id (owner_id)	FK	Lien vers USERS — clé d'isolement
VARCHAR	name		Nom de l'entreprise cliente
VARCHAR	email		Email de contact
VARCHAR	country		Pays — mapping géographique
VARCHAR	default_currency		Devise par défaut (USD, EUR, MAD...)
ENUM	status		ACTIVE, ARCHIVED
TIMESTAMP	created_at		Audit trail
TIMESTAMP	updated_at		Audit trail

Figure 3.2 — Diagramme UML : Entité CLIENTS avec isolement owner_id

3.2.3 Entité CONTRACTS — Cycle de Vie Contractuel

L'entité CONTRACTS est le cœur métier de l'application. Elle maintient le cycle de vie complet d'un contrat freelance via une machine à états (DRAFT → ACTIVE → COMPLETED | CANCELLED).

CONTRACTS			
Type SQL	**Colonne**	**Clé**	**Description**
BIGINT	id	PK	Auto Increment
BIGINT	client_id	FK	Lien vers CLIENTS
VARCHAR	title		Intitulé du contrat
TEXT	description		Périmètre de la mission
DECIMAL(19,4)	total_value		Valeur totale — précision financière

VARCHAR(3)	currency		Code devise ISO 4217
DATE	start_date		Date de début
DATE	deadline		Date d'échéance
ENUM	status		DRAFT, ACTIVE, COMPLETED, CANCELLED
VARCHAR	owner_id		Clé d'isolement multi-tenant
TIMESTAMP	created_at		Audit trail immuable (@CreationTimestamp)

Figure 3.3 — Diagramme UML : Entité CONTRACTS avec machine à états

3.3 Diagramme de Classes UML Complet

Le diagramme de classes UML ci-dessous représente l'ensemble des entités du domaine métier de ClientLedger avec leurs attributs, types de données, et les quatre relations qui les lient : USERS→CLIENTS (possession par owner_id), CLIENTS→CONTRACTS (association Many-to-One via client_id), CONTRACTS→PAYMENTS (composition via contract_id), et la dépendance d'isolement multi-tenant owner_id entre USERS et CONTRACTS (flèche en pointillé rouge).

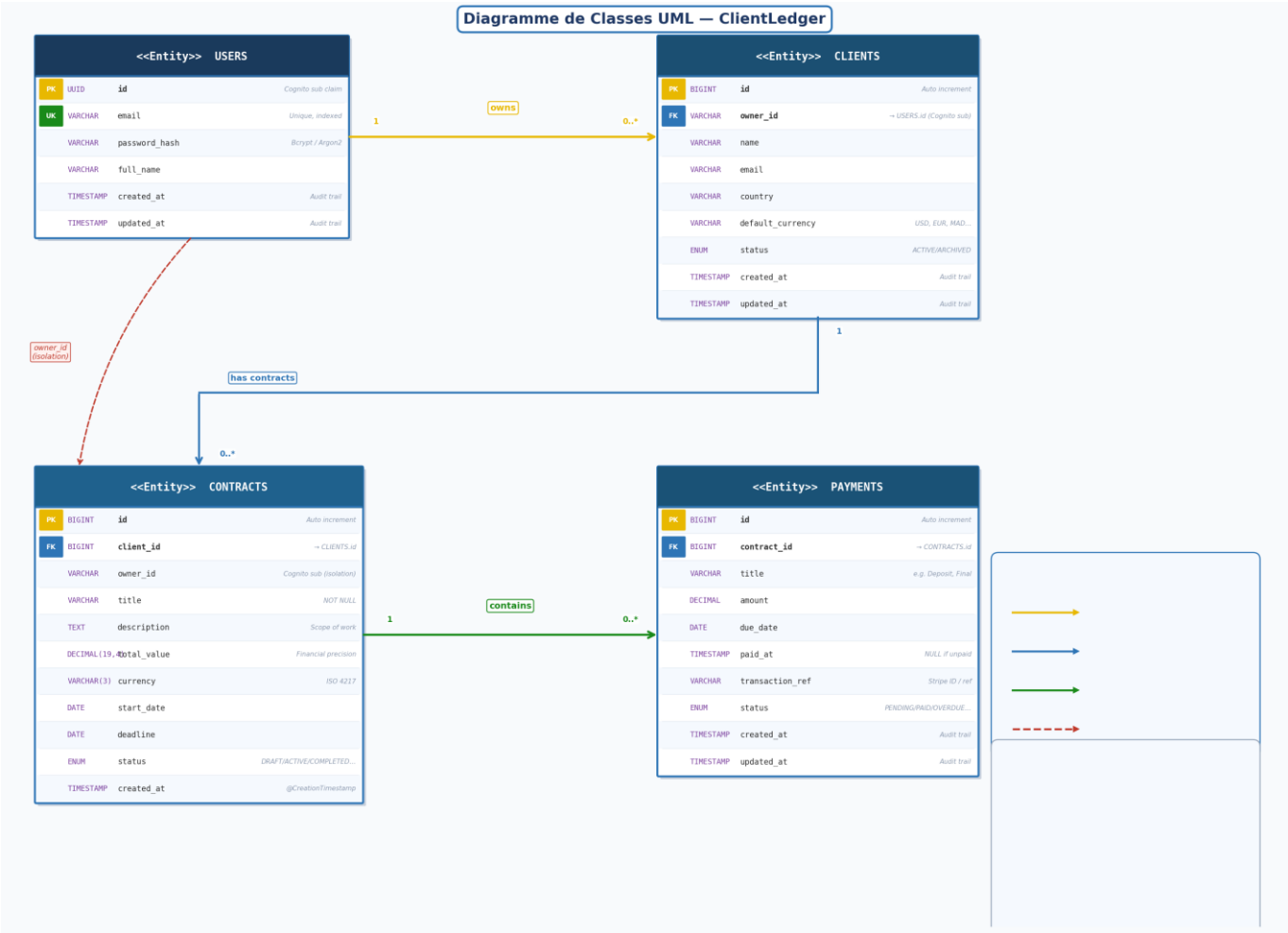


Figure 3.5 — Diagramme de Classes UML complet : entités ClientLedger et relations

Les multiplicités confirment la logique métier : un utilisateur (1) possède zéro ou plusieurs clients (0..*) ; un client (1) a zéro ou plusieurs contrats (0..*) ; un contrat (1) contient zéro ou plusieurs paiements (0..*). La flèche en pointillé rouge (owner_id) garantit que chaque entité porte le Subject ID Cognito du propriétaire, rendant l'accès inter-utilisateurs mathématiquement impossible au niveau des requêtes JPA.

Conclusion

Ce chapitre a modélisé le domaine métier de ClientLedger à travers quatre entités JPA normalisées. Le pattern owner_id implémenté sur CLIENTS et CONTRACTS constitue la garantie mathématique d'isolement multi-tenant : chaque utilisateur ne peut accéder qu'aux données portant son Subject ID Cognito. L'utilisation de DECIMAL(19,4) pour les montants financiers et de @CreationTimestamp pour l'audit trail reflète une conception rigoureuse adaptée à un contexte financier.

Chapitre 4 : Infrastructure Cloud AWS

Introduction

Ce chapitre décrit l'infrastructure AWS de ClientLedger, entièrement provisionnée par Terraform. Il présente la topologie réseau (VPC, Security Groups), les composants de déploiement (Elastic Beanstalk, S3, API Gateway, RDS, Cognito, SES), et le pipeline CI/CD GitHub Actions qui automatise les déploiements en production.

4.1 Topologie de Déploiement

ClientLedger est engineered pour opérer dans un environnement AWS hautement disponible. L'architecture évite les points uniques de défaillance (SPOF) en distribuant les ressources de calcul et de base de données à travers une configuration VPC sécurisée avec des groupes de sécurité strictement configurés.

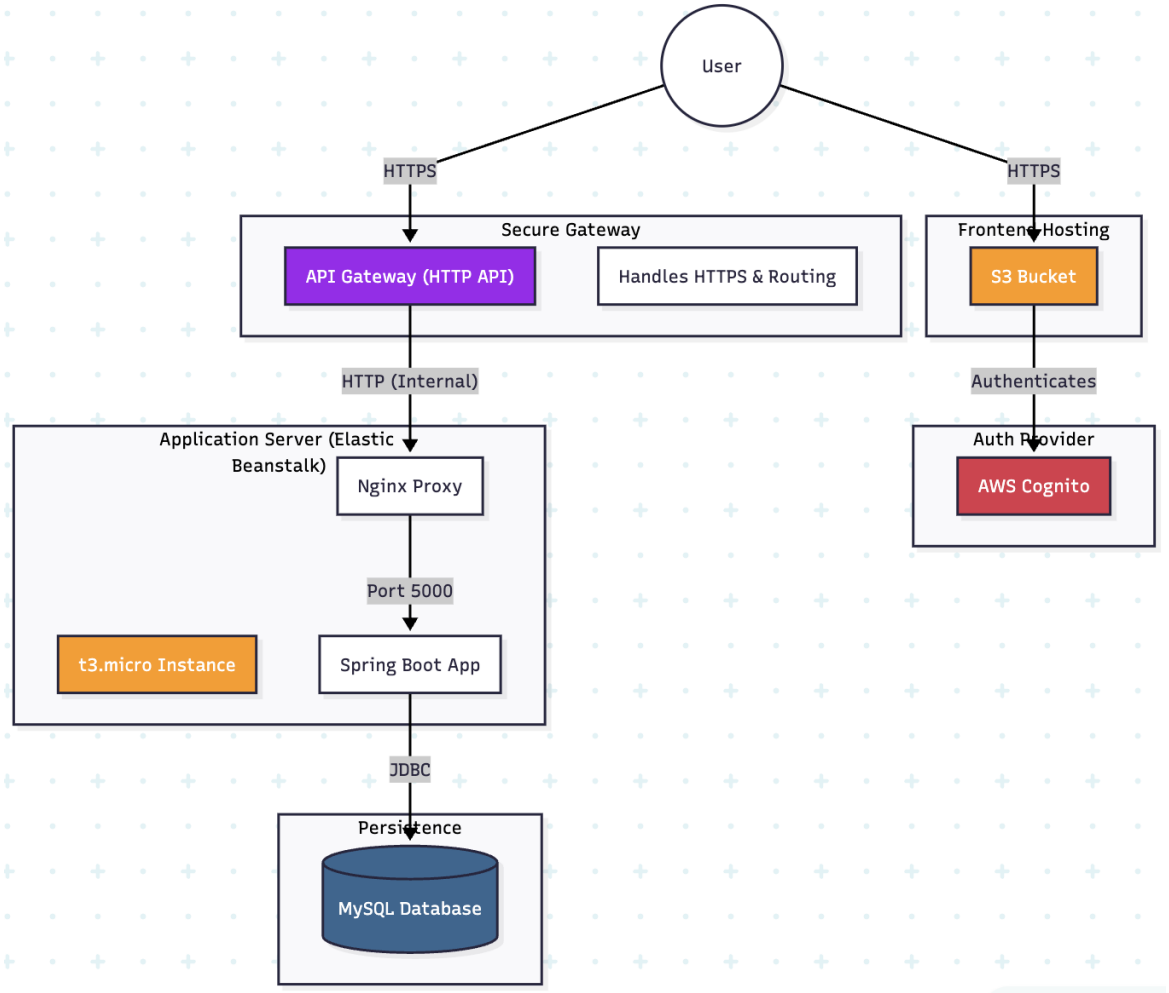


Figure 4.1 — Diagramme de Topologie Cloud AWS de ClientLedger

4.2 Composants AWS Déployés

4.2.1 AWS API Gateway (HTTP API)

L'API Gateway HTTP constitue le point d'entrée unique de l'infrastructure. Il agit comme proxy inverse sécurisé, gérant le routage HTTPS, la terminaison TLS et la redirection vers l'environnement Elastic Beanstalk en HTTP interne. Cette conception évite d'exposer directement les instances EC2 à l'internet public.

4.2.2 AWS Elastic Beanstalk (Backend)

L'application Spring Boot est déployée sur AWS Elastic Beanstalk, qui orchestre automatiquement les ressources EC2 (instance t3.micro), le load balancing, la surveillance et les mises à jour d'applications. Un proxy Nginx est configuré sur l'instance pour transmettre les requêtes vers le port 5000 de la JVM.

4.2.3 Amazon S3 + API Gateway (Frontend)

Le build statique Next.js (export HTML/CSS/JS) est hébergé dans un bucket S3 privé, exposé via une API Gateway distincte avec HTTPS forcé. Cette architecture garantit une haute disponibilité (SLA 99.99% d'AWS S3), des performances optimales via le réseau de distribution d'AWS, et une séparation nette entre les ressources statiques et l'API dynamique.

4.2.4 Amazon RDS (MySQL 8.0)

La persistance des données est assurée par Amazon RDS en mode MySQL 8.0 sur une instance db.t3.micro avec 20 Go de stockage alloué. L'instance RDS est déployée dans un sous-réseau privé du VPC, inaccessible depuis l'internet public.

La connectivité est contrôlée par un Security Group dédié (clientledger-db-sg) qui n'autorise le trafic entrant que sur le port 3306 (MySQL) depuis le Security Group de l'application (clientledger-app-sg). Le code Terraform correspondant est :

```
resource "aws_security_group" "db_sg" {
  ingress {
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.app_sg.id]
  }
```

```
}
```

4.2.5 Amazon SES (Simple Email Service)

Le service Amazon SES est intégré pour l'envoi de notifications email transactionnelles : confirmation de contrat, rappels d'échéance de paiement, et alertes de statut. L'intégration est réalisée via un service Spring Boot dédié (EmailService) qui utilise les credentials IAM configurés dans l'environnement Elastic Beanstalk.

4.3 Pipeline CI/CD avec GitHub Actions

Le déploiement continu est entièrement automatisé via GitHub Actions. Lorsqu'un push est effectué sur la branche principale, le pipeline exécute séquentiellement : la compilation Maven du JAR Spring Boot, les tests unitaires JPA, la génération du build statique Next.js, et le déploiement sur les services AWS cibles via les credentials stockés dans les GitHub Secrets.

- AWS_ACCESS_KEY_ID et AWS_SECRET_ACCESS_KEY : credentials IAM pour le déploiement.
- PROD_ENV_FILE : variables d'environnement de production (URLs Cognito, RDS, etc.).

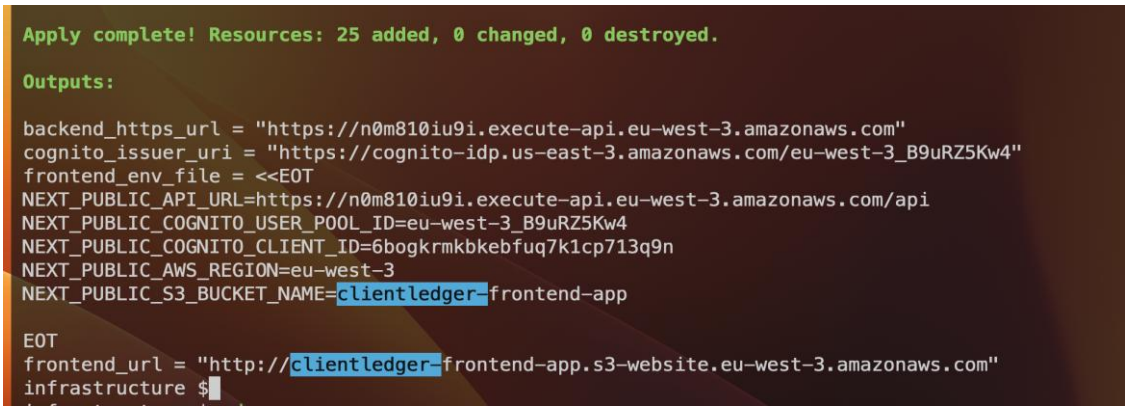


Figure 4.2 — Sorties Terraform après provisioning de l'infrastructure AWS

Chapitre 5 : Cybersécurité & Gestion des Identités (IAM)

5.1 Architecture Zero-Trust

ClientLedger implémente le principe Zero-Trust : aucune requête n'est considérée comme légitime a priori, qu'elle provienne du réseau interne ou externe. Chaque appel API doit présenter un token JWT cryptographiquement valide, vérifié par signature RSA asymétrique à chaque requête, sans exception.

Ce modèle s'oppose au modèle "castle-and-moat" traditionnel qui fait confiance au trafic interne une fois la périmètre franchie. Le Zero-Trust est particulièrement adapté aux architectures cloud où les périmètres réseau sont fluides et où les menaces internes (compromission d'un microservice, d'une instance EC2) sont aussi critiques que les menaces externes.

5.2 Authentification Déléguée via AWS Cognito

La gestion des identités est entièrement déléguée à AWS Cognito User Pools. Cette stratégie confère à ClientLedger des capacités de sécurité de niveau entreprise sans la complexité et les risques associés à la gestion locale de credentials :

- Protection anti-brute-force native avec blocage automatique des tentatives répétées.
- Rotation automatique des credentials et politique de mots de passe configurable.
- Émission de tokens JWT signés RS256 avec durée de vie configurable.
- Support OAuth 2.0 / OpenID Connect pour une intégration standardisée.
- Scopes d'autorisation granulaires (contracts.create, contracts.read).

Le User Pool Cognito est configuré via Terraform avec des flux d'authentification explicitement définis : ALLOW_USER_SRP_AUTH (Secure Remote Password), ALLOW_USER_PASSWORD_AUTH et ALLOW_REFRESH_TOKEN_AUTH. Le client frontend utilise le flux implicite OAuth 2.0 avec les scopes openid, email, profile et les scopes applicatifs métier.

5.3 Vérification JWT & JWKS

Le backend Spring Boot implémente un filtre de sécurité OAuth2 Resource Server qui intercepte chaque requête entrante protégée. Le processus de

validation JWT suit un protocole cryptographique rigoureux en plusieurs étapes :

- Extraction du Key ID (kid) depuis l'en-tête du token JWT.
- Récupération dynamique des clés publiques RSA depuis le endpoint JWKS de Cognito : https: //.../cognito-idp.{region}.amazonaws.com/{userPoolId}/.well-known/jwks.json
- Correspondance du kid avec la clé publique appropriée dans le JWKS.
- Vérification cryptographique de la signature RSA du token.
- Validation de la date d'expiration (exp), de l'audience (aud) et de l'émetteur (iss).
- Extraction du Subject claim (sub) pour l'injection dans le contexte @AuthenticationPrincipal.

La configuration Spring Security correspondante (SecurityConfig.java) définit une chaîne de filtres qui : désactive CSRF (architecture stateless), configure CORS avec origines whitelistées, protège tous les endpoints /api/contracts/** avec authentification JWT, et autorise publiquement uniquement les endpoints de santé.

5.4 Isolement Multi-Tenant

L'isolement des données entre les utilisateurs est la garantie de sécurité la plus critique d'une application SaaS multi-tenant. ClientLedger implémente cette isolation au niveau de la couche JPA via un mécanisme d'owner_id cryptographique.

Chaque entité métier (Client, Contract) possède une colonne owner_id non nullable, peuplée automatiquement lors de la création avec le Subject ID (sub) extrait du JWT via le contexte Spring Security @AuthenticationPrincipal. Toutes les requêtes JPA sont filtrées sur cette colonne, rendant mathématiquement impossible l'accès aux données d'un autre utilisateur, même en cas de manipulation d'identifiants numériques.

Ce pattern est supérieur à un contrôle d'accès basé uniquement sur les rôles (RBAC) car il opère au niveau de la donnée et non au niveau de l'endpoint, offrant une protection contre les vulnérabilités d'Insecure Direct Object Reference (IDOR).

5.5 Tableau de Synthèse des Contrôles de Sécurité

Contrôle de Sécurité	Description	Criticité
JWT RS256 Validation	Vérification cryptographique de chaque requête via JWKS Cognito	**Élevé**
Multi-Tenant Row Isolation	Filtre owner_id sur toutes les requêtes JPA	**Critique**
TLS/HTTPS Enforced	Chiffrement en transit sur toutes les couches	**Élevé**
VPC Private Subnet (RDS)	Base de données inaccessible depuis l'internet public	**Critique**
Security Group Whitelisting	Port 3306 autorisé uniquement depuis l'IP de l'application	**Élevé**
No Password Storage	Authentification déléguée à AWS Cognito — aucun hash local	**Critique**
Brute-Force Protection	Géré nativement par Cognito User Pool policies	**Moyen**
CORS Policy	Origines whitelistées, méthodes HTTP restreintes	**Moyen**

Tableau 5.1 — Matrice des Contrôles de Sécurité Implémentés

Conclusion

Ce chapitre a démontré que ClientLedger implémente une architecture de sécurité multicouches robuste. La délégation IAM à AWS Cognito élimine les risques de stockage local de credentials. La validation JWT RS256 via JWKS assure que chaque requête est cryptographiquement authentifiée. L'isolement multi-tenant par owner_id protège contre les attaques IDOR. L'ensemble de ces contrôles forme une défense en profondeur alignée sur les standards NIST et OWASP.

Chapitre 6 : Architecture Applicative & API REST

Introduction

Ce chapitre présente l'organisation interne du backend Spring Boot selon une architecture en couches (Controller / Service / Repository / Entity / Security), détaille le catalogue complet des endpoints REST exposés, et décrit les services transversaux (gestion des exceptions, génération PDF, statistiques financières).

6.1 Architecture en Couches du Backend

Le backend Spring Boot est organisé selon une architecture en couches (Layered Architecture) qui sépare clairement les responsabilités et facilite la testabilité individuelle de chaque couche :

- Couche Controller : réception des requêtes HTTP, validation des paramètres, sérialisation JSON.
- Couche Service : logique métier, orchestration des opérations, appels aux repositories.
- Couche Repository (JPA) : abstraction de la couche de persistance, requêtes JPQL.
- Couche Entity : mapping objet-relationnel via annotations JPA/Hibernate.
- Couche Security : filtre JWT, extraction du contexte utilisateur.

6.2 Endpoints de l'API REST

L'API REST de ClientLedger expose des endpoints CRUD sécurisés pour chaque ressource métier. Tous les endpoints /api/** requièrent un JWT valide dans le header Authorization: Bearer {token}.

Méthode	Endpoint	Rôle	Auth
GET	/health	Vérification de santé applicative	Public
GET	/api/clients	Liste des clients du freelancer	JWT
POST	/api/clients	Créer un nouveau client	JWT
GET	/api/clients/{id}	Détails d'un client spécifique	JWT
DELETE	/api/clients/{id}	Supprimer un client	JWT
GET	/api/contracts	Liste des contrats actifs	JWT

POST	/api/contracts	Créer un nouveau contrat	JWT
PATCH	/api/contracts/{id}	Mettre à jour le statut du contrat	JWT
GET	/api/stats/summary	KPIs financiers du tableau de bord	JWT

Tableau 6.1 — Catalogue des Endpoints API REST de ClientLedger

6.3 Gestion des Exceptions Globale

Le backend intègre un GlobalExceptionHandler Spring (annotation @ControllerAdvice) qui intercepte les exceptions métier et les transforme en réponses HTTP sémantiquement correctes (404 Not Found, 403 Forbidden, 422 Unprocessable Entity), évitant l'exposition de stack traces techniques dans les réponses d'erreur API.

6.4 Génération de PDF Contractuels

Le service PdfService intégré au module contract permet la génération automatique de contrats en format PDF directement depuis l'application. Cette fonctionnalité permet aux freelancers d'envoyer des contrats professionnels à leurs clients directement depuis le dashboard, consolidant ainsi le workflow de gestion contractuelle en une seule plateforme.

Conclusion

Ce chapitre a présenté l'architecture applicative du backend Spring Boot de ClientLedger. L'organisation en couches garantit une séparation claire des responsabilités et facilite la testabilité de chaque composant. Le catalogue de 9 endpoints REST couvre l'ensemble des opérations CRUD nécessaires au cycle de vie des contrats freelance. Les services transversaux (exceptions, PDF, stats, email) enrichissent l'expérience utilisateur tout en maintenant une architecture cohérente et maintenable.

Chapitre 7 : Réalisation

Introduction

Ce chapitre présente la réalisation concrète de ClientLedger à travers les interfaces utilisateurs développées. Il documente chaque écran de l'application, depuis la page d'accueil publique jusqu'au dashboard de gestion financière, en illustrant la cohérence entre les choix architecturaux des chapitres précédents et leur implémentation visible dans le produit final

7.1 Page d'Accueil — Landing Page

La page d'accueil de ClientLedger constitue le premier point de contact avec les utilisateurs potentiels. Elle adopte un design sombre et moderne, aligné sur les codes visuels du secteur tech-fintech, avec la signature visuelle « Freelance Ops Perfected. » comme accroche principale.

La barre de navigation comprend les sections Features, Security et Pricing, ainsi que les boutons Login et Get Started. Le tagline secondaire « Automate contracts, monitor global revenue, and secure your financial future. Built for the modern developer-turned-founder. » positionne clairement le produit pour son audience cible. Un bouton d'appel à l'action « Join the Private Beta » oriente les visiteurs vers l'inscription, tandis que le bouton « Live Demo » permet une découverte immédiate.

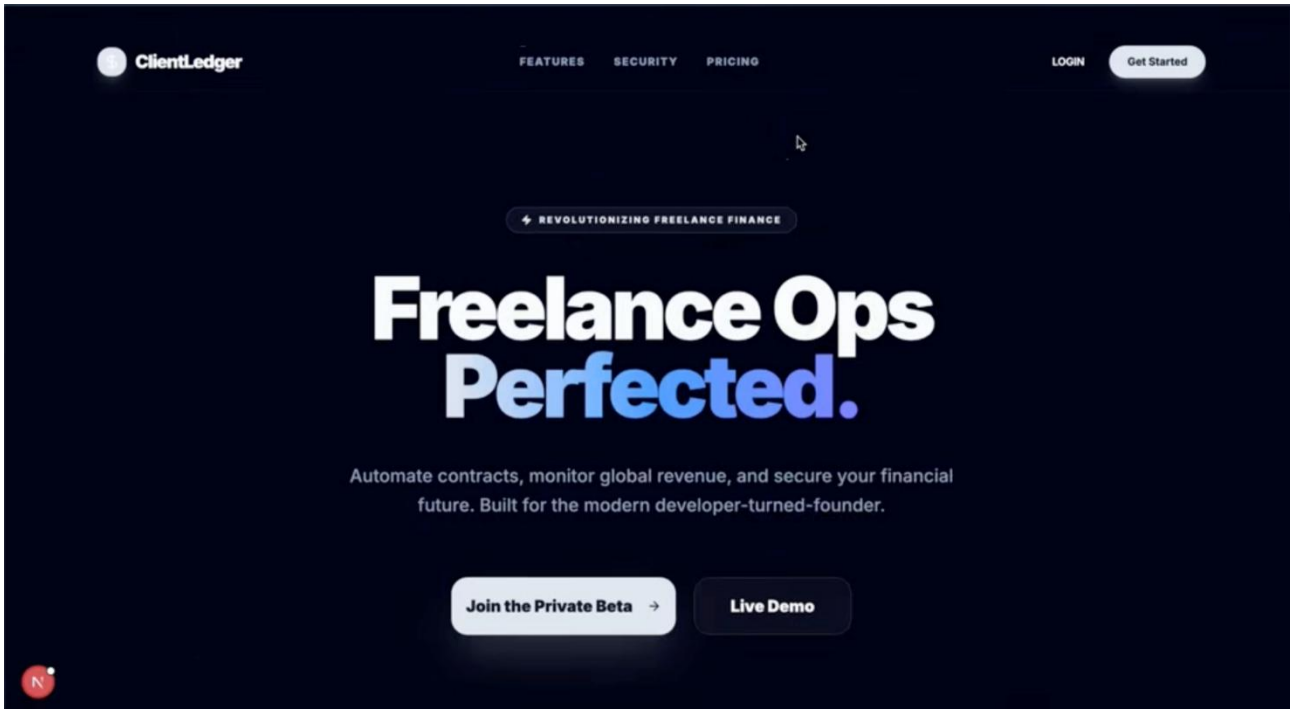


Figure 1:Page d'accueil de ClientLedger : accroche et navigation principale

7.2 Page de Tarification

La page de tarification présente le modèle commercial de ClientLedger avec le titre « Scalable Value. » et le sous-titre « Whether you're freelancing in Casablanca or Berlin, we have a plan to help you

hit your 35-millionaire goal. » Cette accroche internationale souligne la vocation globale de la plateforme.

Deux plans sont proposés pour adresser différents profils de freelancers :

Plan	Tarif	Fonctionnalités
Starter Beta	$0 / mois	Jusqu'à 5 clients actifs, Multi-Currency (MAD, EUR, USD), AWS SES Email, Standard PDF Invoices
Pro Ledger	$15 / mois	Clients illimités, Advanced Revenue Analytics, Custom Branding & Profiles, Priority PDF Generation

Tableau 7.1 — Plans tarifaires de ClientLedger

Le plan Starter Beta à $0/mois permet aux nouveaux freelancers d'adopter la plateforme sans risque financier. Le plan Pro Ledger à $15/mois déverrouille les fonctionnalités avancées pour les freelancers établis. Cette stratégie freemium maximise l'acquisition d'utilisateurs et facilite leur conversion vers le plan payant.

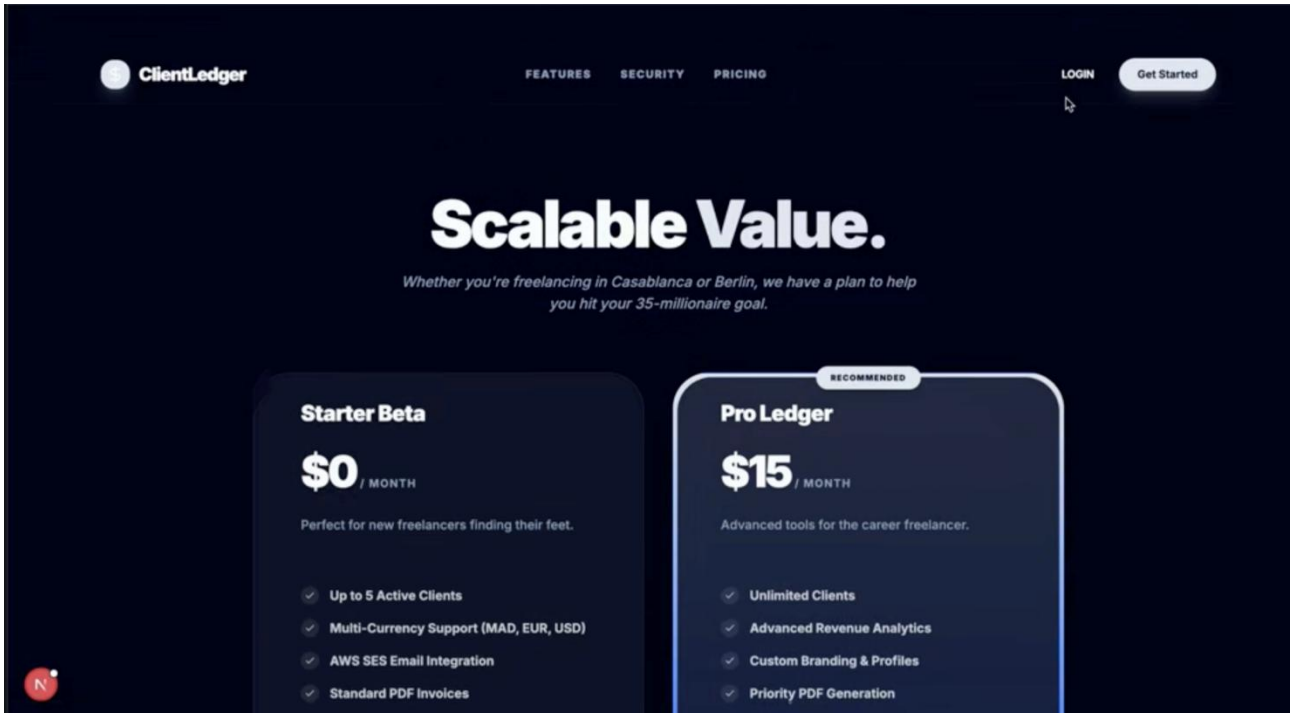


Figure 2:Page de tarification : plans Starter Beta ($0) et Pro Ledger ($15/mois)

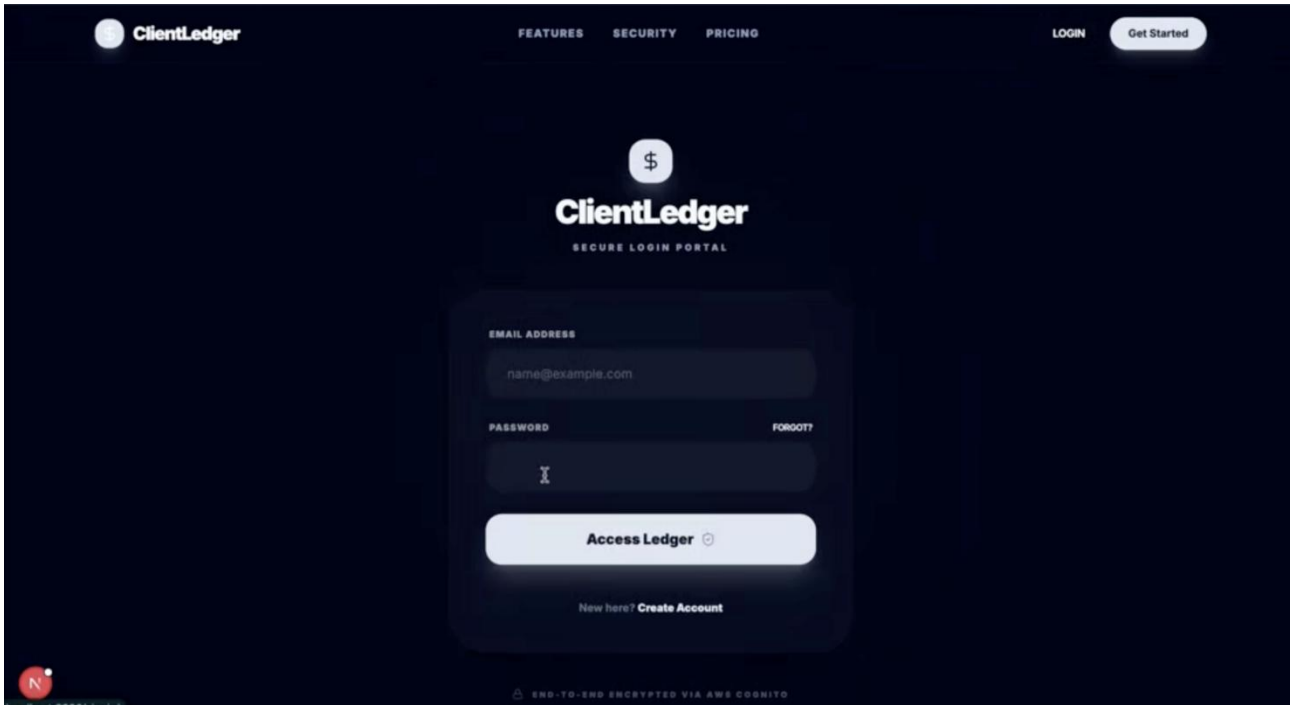


Figure 3:Portail de connexion sécurisé avec authentification AWS Cognito

7.3 Portail de Connexion Sécurisée

Le portail de connexion affiche clairement le badge « SECURE LOGIN PORTAL » et la mention « END-TO-END ENCRYPTED VIA AWS COGNITO » en pied de page, renforçant la confiance des utilisateurs dans la sécurité de leurs données. Le formulaire collecte l'adresse email et le mot de passe, avec une option « Forgot? » pour la réinitialisation.

L'authentification est entièrement déléguée à AWS Cognito via le flux ALLOW_USER_PASSWORD_AUTH. Les credentials de l'utilisateur ne transitent jamais par les serveurs de l'application — ils sont envoyés directement au endpoint Cognito. En cas d'authentification réussie, Cognito retourne un triplet de tokens (access_token, id_token, refresh_token). Le frontend stocke l'access_token en mémoire (non dans localStorage) pour éviter les attaques XSS.

7.4 Dashboard Financier Principal

Le tableau de bord est le cœur opérationnel de ClientLedger. Il agrège en temps réel les indicateurs financiers clés du freelancer, récupérés via l'endpoint sécurisé /api/stats/summary. Les quatre KPIs affichés en en-tête sont :

- Total Revenue : montant cumulé des paiements reçus (ex. $159,700 avec variation +12% vs mois précédent).
- Pending : montant des contrats et paiements en attente (ex. $10,000 — Draft or Active Contracts).
- Active Clients : nombre total de clients actifs dans le portefeuille (ex. 5 — Total Client Base).
- Total Overdue : montant des paiements en retard non encaissés (ex. $13,000 — Unpaid Past Due Date).

Le graphique « Revenue Trends » présente l'évolution mensuelle (Jan à Déc) des contrats payés sous forme de barres chronologiques, avec une info-bulle affichant le détail par période (ex. Jul 2025 : $3,500). Le graphique « Financial Health » présente un diagramme circulaire Paid vs. Unpaid

donnant une vue immédiate du taux de recouvrement. Une notification d'accueil contextuelle « Welcome back! Signing into your secure ledger... » s'affiche lors de la connexion.

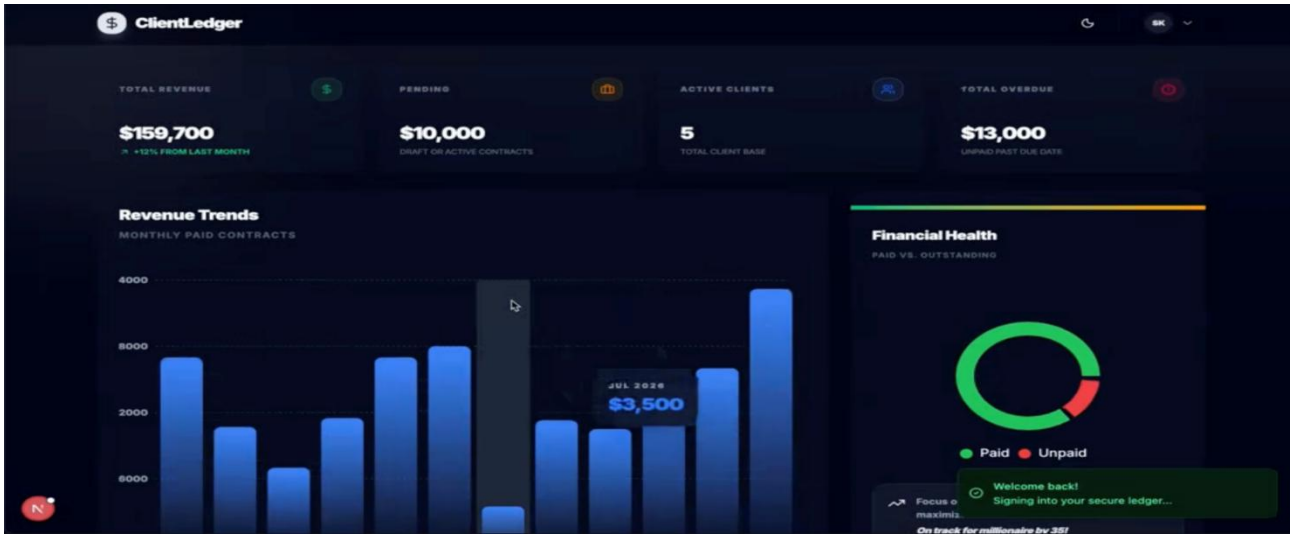


Figure 4:Dashboard financier : KPIs, Revenue Trends et Financial Health

7.5 Gestion des Contrats

L'interface de gestion des contrats constitue la fonctionnalité centrale de ClientLedger. Le panneau latéral gauche liste tous les clients avec leur indicatif pays (TR, CA, GE, DU, LO) et leur nom, permettant une navigation rapide entre les portefeuilles. La zone principale affiche une vue consolidée de tous les contrats du client sélectionné.

Les filtres de statut (ALL / DRAFT / PENDING / PAID) et les deux modes d'affichage (grille et liste) permettent une navigation rapide. La barre de recherche « Search by title... » permet de filtrer les contrats par mot-clé. Chaque carte affiche le titre du contrat, la valeur totale avec la devise ISO, un badge de statut coloré, et deux actions rapides :

- Download : déclenche la génération PDF via le PdfService et télécharge le document.
- Email : envoie le contrat PDF au client via le EmailService (Amazon SES).

Le pied de tableau affiche le nombre total de résultats (ex. 27 TOTAL) et la pagination (PAGE 1/3). Des exemples de contrats visibles dans l'interface incluent : Frontend Refactor (4,500 MAD — PENDING), Q4 Server Costs (1,200 GBP — PAID), Full Stack Web App (22,000 USD — PAID), Cybersecurity Retainer (5,000 GBP — OVERDUE), Payment Gateway Fix (3,500 USD — PAID).

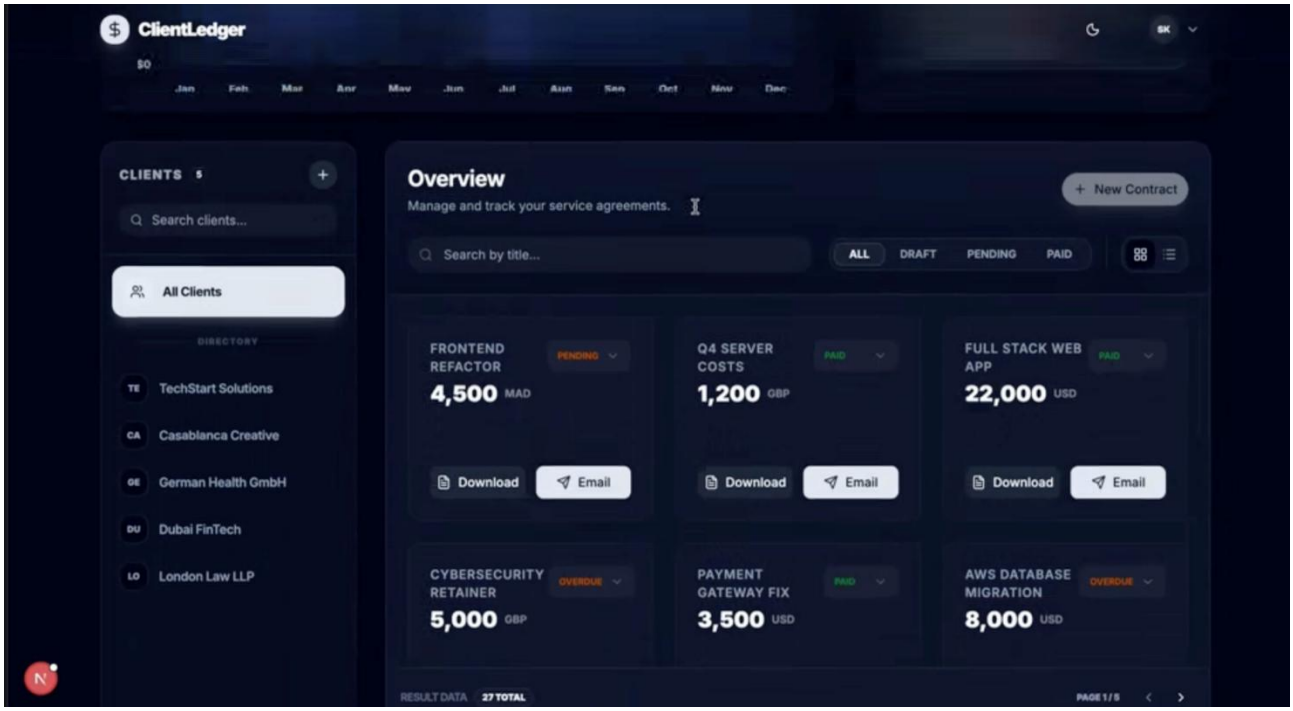


Figure 5:nterface de gestion des contrats avec filtres, badges de statut et pagination

7.6 Création d'un Nouveau Contrat

La création d'un contrat s'effectue via une modale contextuelle déclenchée par le bouton « + New Contract ». Le formulaire contextuel s'intitule « New Contract for [Nom du Client] » (ex. New Contract for TechStart Solutions) et collecte :

- Contract Title : titre descriptif de la mission (ex. Security Check, AWS Migration).
- Total Value : montant numérique du contrat (ex. 199).
- Currency : devise sélectionnée parmi les options du client (par défaut la devise client).

À la soumission via le bouton « Create Contract », le backend crée l'entité CONTRACTS avec le statut initial DRAFT, associée au client_id du client sélectionné et au owner_id extrait du JWT de l'utilisateur courant. Le contrat est immédiatement visible dans la grille avec le badge DRAFT. La validation côté serveur garantit que la valeur est positive et que la devise est un code ISO 4217 valide.

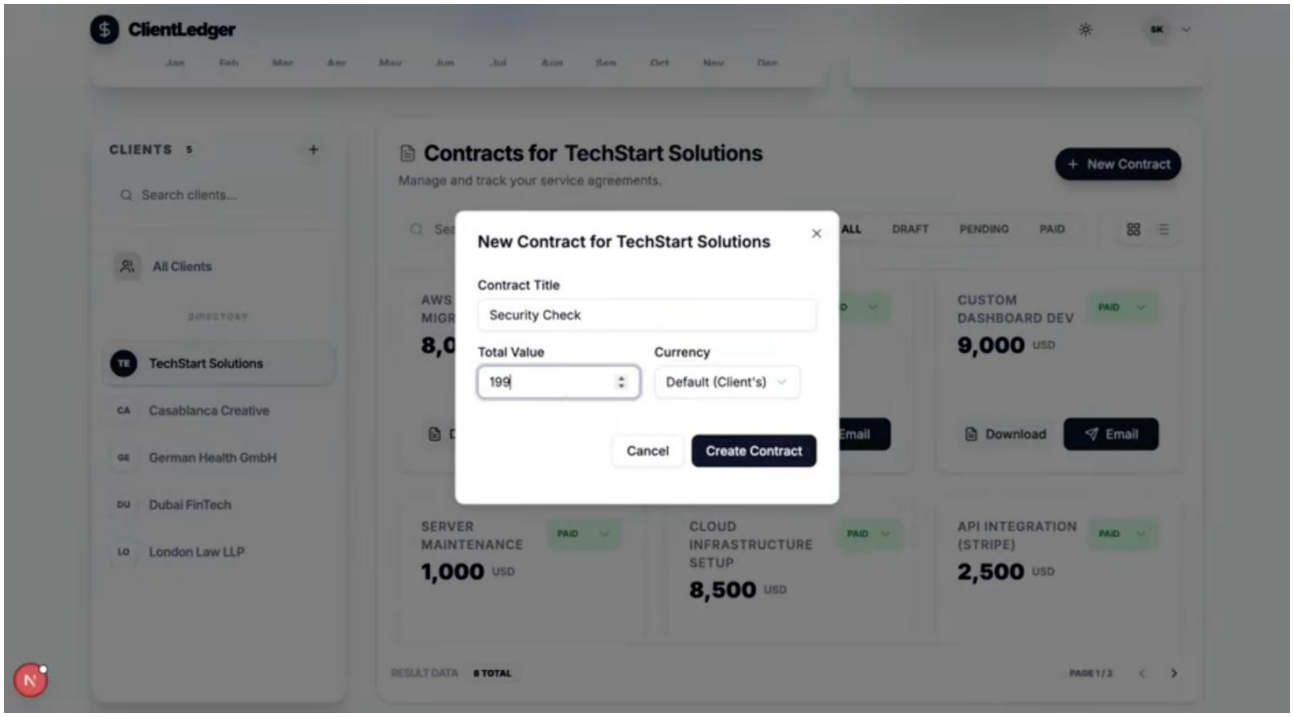


Figure 6:Modale de création d'un nouveau contrat avec validation de formulaire

7.7 Paramètres et Profil de l'Entreprise

La page Settings permet au freelancer de configurer son profil professionnel qui apparaîtra sur toutes les factures PDF générées. Les champs configurables sont :

- Company Logo : upload d'un logo personnalisé (les initiales sont utilisées comme placeholder, ex. CL).
- Company Legal Name : raison sociale officielle (ex. ClientLedger Ops).
- Tax / VAT ID : numéro fiscal pour les factures (ex. TAX-999).
- Phone Number : numéro de contact international (ex. +212 600-000000).
- Business Address : adresse affichée dans la section From des factures (ex. Casablanca Tech Hub).

La mention « This address will be displayed in the From section of your invoices. » guide l'utilisateur sur l'utilisation de ces données. Un second onglet « Billing (Coming Soon) » anticipe les futures fonctionnalités de gestion des abonnements. Toutes les modifications sont sauvegardées via un appel API PATCH sécurisé par JWT.

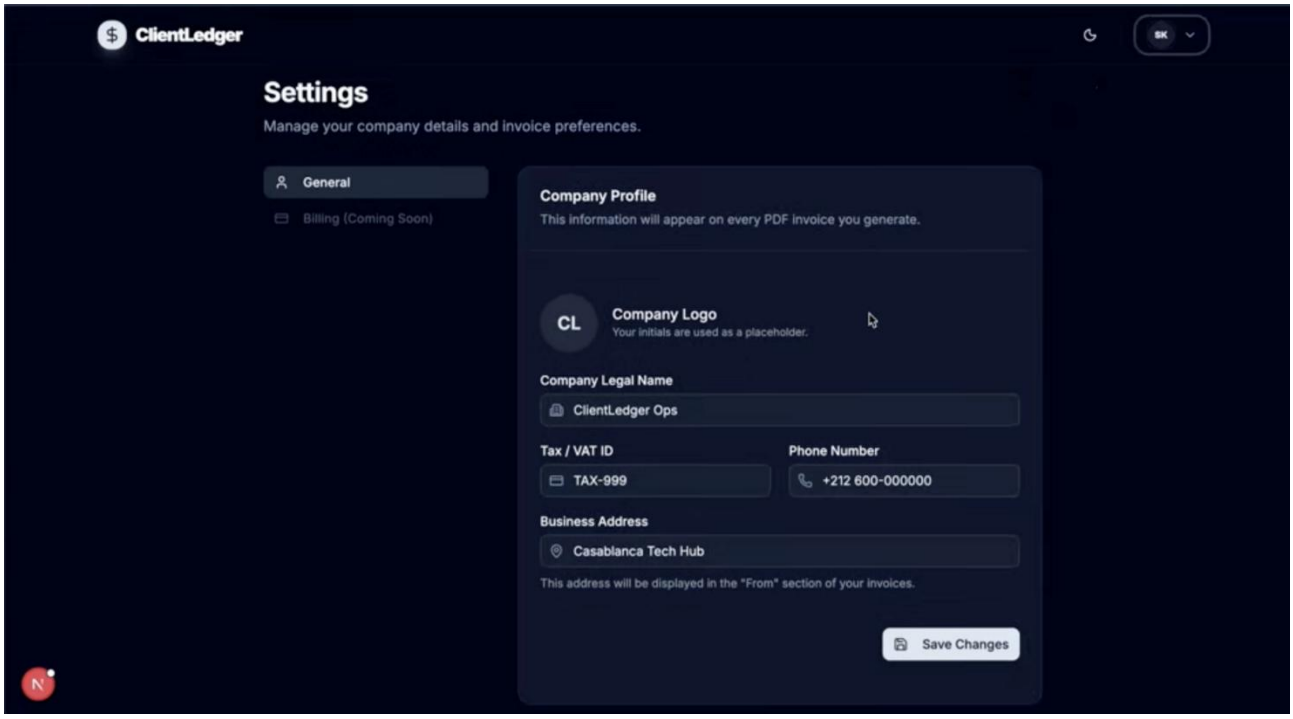


Figure 7:Page de paramètres : configuration du profil entreprise et des informations de facturation

7.8 Génération et Aperçu de Factures PDF



Figure 8:Aperçu d'une facture PDF générée par ClientLedger

Conclusion

Ce chapitre a présenté l'ensemble des interfaces réalisées dans ClientLedger : landing page, page de tarification, portail d'authentification, dashboard financier, gestion des contrats, création de contrat, paramètres de profil, et

génération de PDF. La cohérence visuelle et fonctionnelle de ces huit interfaces démontre la maturité de l'implémentation et valide la convergence entre les choix architecturaux documentés dans les chapitres précédents et le produit final livré.

Chapitre 8 : Analyse, Évaluation & Perspectives

Introduction

Ce chapitre final propose une analyse critique des forces et limitations de ClientLedger, une évaluation objective des objectifs atteints, et une feuille de route prospective pour les évolutions futures : intégration Stripe, AWS WAF, observabilité CloudWatch/X-Ray.

8.1 Analyse des Forces du Projet

ClientLedger démontre une maîtrise cohérente des pratiques d'ingénierie cloud modernes. Les points forts architecturaux identifiés sont :

- Infrastructure as Code : la reproductibilité totale de l'environnement via Terraform élimine les erreurs de configuration manuelle et facilite la création d'environnements identiques (dev/staging/prod).
- Séparation des préoccupations : le découplage strict frontend/backend permet une évolution indépendante de chaque couche sans impact sur l'autre.
- Délégation IAM à AWS Cognito : cette stratégie élimine les risques de stockage et de gestion locale de credentials, transférant cette responsabilité à un service managé certifié SOC 2 Type II.
- Isolation multi-tenant cryptographique : le pattern owner_id offre une garantie mathématique d'isolation des données, supérieure aux approches basées uniquement sur les politiques d'accès.

8.2 Feuille de Route Future

7.2.1 Infrastructure de Paiement Automatisée (Stripe)

La prochaine phase de développement implique la transition de ClientLedger d'un outil de suivi passif à un processeur de paiement actif via l'intégration de l'API Stripe. Pour maintenir une posture de sécurité stricte et réduire le périmètre de conformité PCI-DSS, l'application utilisera Stripe Checkout (redirection sécurisée) et les Webhooks Stripe pour la validation des paiements. Le backend Spring Boot sera étendu pour valider les signatures cryptographiques des webhooks Stripe (header Stripe-Signature) et transitionner automatiquement les statuts de contrats de PENDING à PAID.

8.2.2 Atténuation des Menaces Avancées (AWS WAF & GuardDuty)

À mesure que la plateforme évolue, la surface d'attaque s'élargit inévitablement. Les itérations architecturales futures intégreront AWS Web

Application Firewall (WAF) au niveau de l'Application Load Balancer pour atténuer les botnets automatisés et les attaques DDoS Layer 7. L'exploitation d'AWS CloudTrail et d'Amazon GuardDuty fournira une détection automatisée des anomalies comportementales, alertant les administrateurs sur des patterns de traversée API suspects ou des tentatives de connexion géographiquement anormales.

8.2.3 Observabilité & SRE

L'intégration d'une stack d'observabilité complète est planifiée : Amazon CloudWatch pour les métriques et les alarmes, AWS X-Ray pour le tracing distribué des requêtes à travers les couches applicatives, et un dashboard CloudWatch centralisé exposant les KPIs opérationnels clés (latence p95, taux d'erreur 5xx, utilisation CPU/mémoire).

Conclusion

Ce chapitre a proposé une analyse critique honnête de ClientLedger, identifiant ses forces (IaC, délégation IAM, isolation multi-tenant, CI/CD) et ses limitations (SPOF, absence de cache, monitoring insuffisant). La feuille de route établie — Stripe, WAF, GuardDuty, observabilité — trace une trajectoire crédible vers une plateforme SaaS de niveau production, démontrant la maturité de la réflexion architecturale portée par ce projet.

Conclusion Générale

Le développement de ClientLedger a permis de démontrer concrètement l'application des principes d'architecture cloud-native et de sécurité Zero-Trust dans le cadre d'un projet académique à portée professionnelle. Ce rapport documente l'ensemble des choix techniques, architecturaux et de sécurité qui ont guidé la conception et le déploiement de cette plateforme SaaS.

Sur le plan de l'ingénierie cloud, ClientLedger illustre comment Terraform permet de codifier une infrastructure AWS complexe de manière reproductible et versionnable, transformant le déploiement d'un processus manuel et risqué en une opération automatisée et idempotente. Le pipeline CI/CD GitHub Actions assure une livraison continue avec zéro temps d'arrêt pour les utilisateurs.

Sur le plan de la cybersécurité, ce projet démontre que les meilleures pratiques de sécurité ne sont pas opposées à la rapidité de développement : la délégation de l'authentification à AWS Cognito, la validation JWT asymétrique, et l'isolement multi-tenant par owner_id constituent un système de défense en profondeur robuste, obtenu sans complexité architecturale excessive.

En définitive, ClientLedger constitue un prototype viable d'un outil SaaS de gestion freelance, et représente une démonstration pratique de la convergence entre le génie logiciel et la cybersécurité, deux disciplines que ce Bachelor réunit dans une formation unique et complémentaire.

Bibliographie & Références

Documentation Officielle

- Amazon Web Services. (2024). Amazon Cognito Developer Guide. https://docs.aws.amazon.com/cognito/
- Amazon Web Services. (2024). AWS Elastic Beanstalk Developer Guide. https://docs.aws.amazon.com/elasticbeanstalk/
- Amazon Web Services. (2024). Amazon RDS User Guide for MySQL. https://docs.aws.amazon.com/rds/
- HashiCorp. (2024). Terraform AWS Provider Documentation. https://registry.terraform.io/providers/hashicorp/aws/
- Spring.io. (2024). Spring Boot Reference Documentation 3.x. https://docs.spring.io/spring-boot/
- Spring.io. (2024). Spring Security OAuth2 Resource Server. https://docs.spring.io/spring-security/
- Next.js Team. (2024). Next.js Documentation v14. https://nextjs.org/docs

Articles & Standards

- RFC 7519 — JSON Web Token (JWT). Internet Engineering Task Force (IETF). https://tools.ietf.org/html/rfc7519
- RFC 7517 — JSON Web Key (JWK). Internet Engineering Task Force (IETF). https://tools.ietf.org/html/rfc7517
- NIST SP 800-207 — Zero Trust Architecture. National Institute of Standards and Technology. (2020).
- OWASP Top Ten 2023. Open Web Application Security Project. https://owasp.org/www-project-top-ten/
- Cloud Security Alliance. (2023). Security Guidance for Cloud Computing Critical Areas of Focus.

Dépôt du Projet

Khalis, M. & Farhoun, I. (2025). ClientLedger — Source Code Repository. GitHub. https://github.com/Admiral-Simo/ClientLedger